



Module 1

Python Basics and Data Processing

Presentation prepared with AI support



Chapter 1

Introduction to Python

Why Python?

Most Popular Language 2025

Python holds a leading position in the TIOBE Index, surpassing other programming languages due to its versatility and simplicity.

Versatile Applications

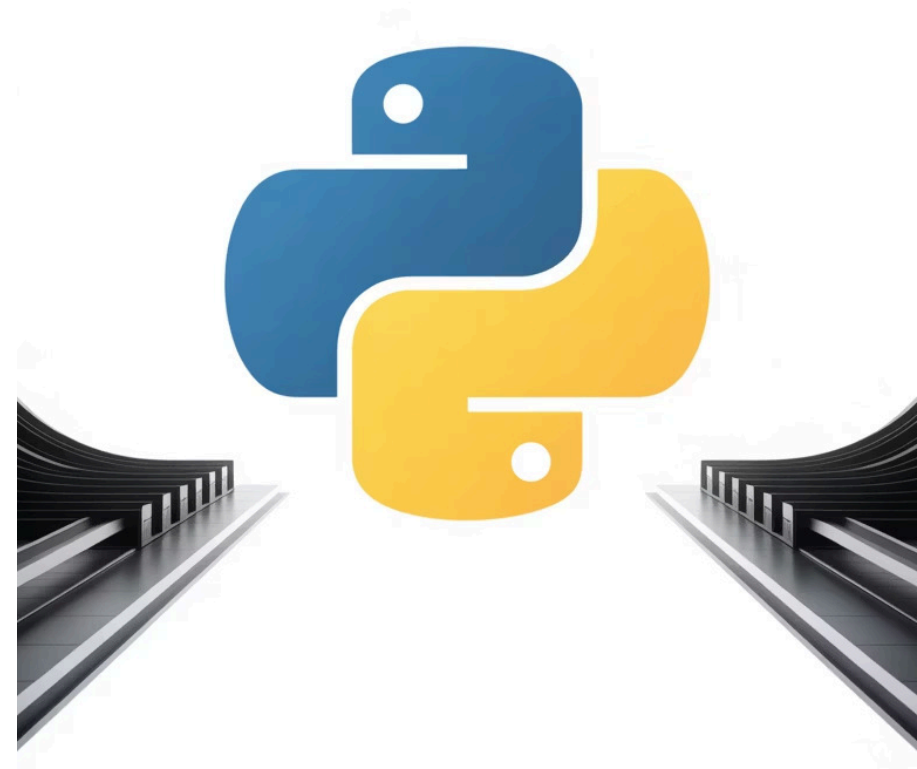
From web development, through data analysis, machine learning, to process automation – Python excels everywhere.

Ideal for Beginners

Its readable syntax, similar to natural language, makes learning Python quick and intuitive.

Python – the language that changes the world

From simple scripts to advanced artificial intelligence systems



Installation and Development Environment

Downloading Python

Visit the official website **python.org** and download the latest stable version of Python 3.x. The installer will automatically configure the interpreter on your system.

Development Environments

- **Jupyter Notebook** – an interactive environment ideal for learning and experiments
- **Visual Studio Code** – a lightweight editor with extensive extensions
- **PyCharm** – a professional IDE with advanced tools



📌 **Environment Management:** Virtualenv and Anaconda allow you to create isolated project environments, which prevents conflicts between packages.

First Python Program

Classic Start

```
print("Hello, World!")
```

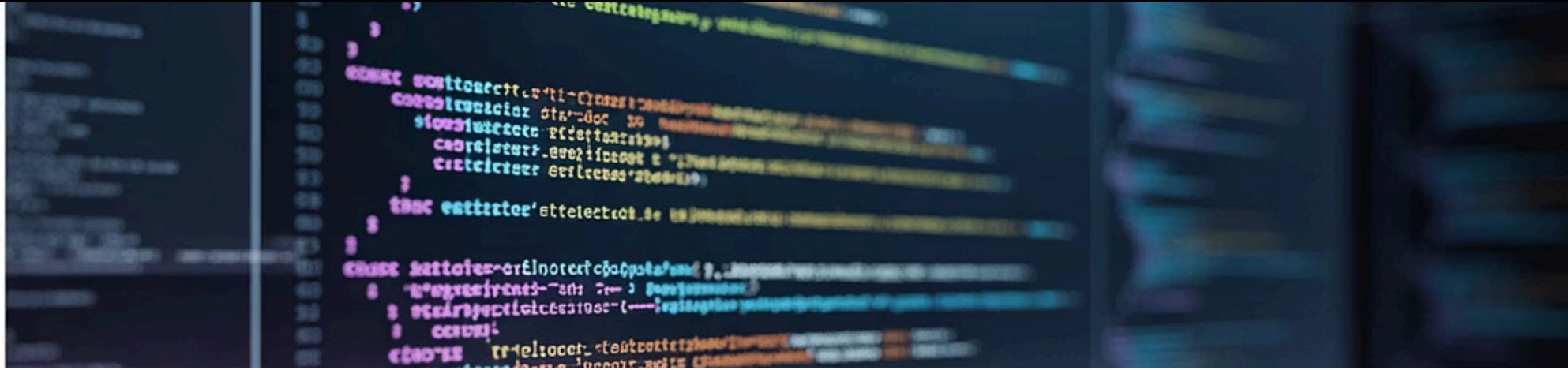
This is the traditional first program for every programmer. The `print()` function displays text on the screen.

Interactive Mode vs Scripts

Interactive mode allows you to test code line by line in the Python console. **Scripts** are .py files containing a complete program.

Running Code

In the terminal: `python script.py`
In a notebook: run the cell with Shift+Enter



Chapter 2

Syntax Fundamentals and Data Types

Variables and Data Types

Python uses **dynamic typing**, which means that the type of a variable is determined automatically when a value is assigned.

int – Integers

```
age = 25  
year = 2025
```

float – Floating-point Numbers

```
price = 19.99  
temperature = -5.5
```

str – Strings

```
name = "Anna"  
text = 'Python'
```

bool – Boolean Values

```
is_active = True  
has_access = False
```

Type Conversion: `int("123")`, `str(456)`, `float("3.14")`

Operators in Python

Arithmetic

- `+` addition
- `-` subtraction
- `*` multiplication
- `/` division
- `//` integer division
- `%` modulo
- `**` exponentiation

Comparison

- `==` equal to
- `!=` not equal to
- `>` greater than
- `<` less than
- `>=` greater than or equal to
- `<=` less than or equal to

Logical

- `and` conjunction (and)
- `or` disjunction (or)
- `not` negation (not)

Example:

`x > 5 and x < 10`

Conditional Statements and Loops

1

if, elif, else

```
if temperature > 25:  
    print("Hot")  
elif temperature > 15:  
    print("Warm")  
else:  
    print("Cold")
```

2

For Loop

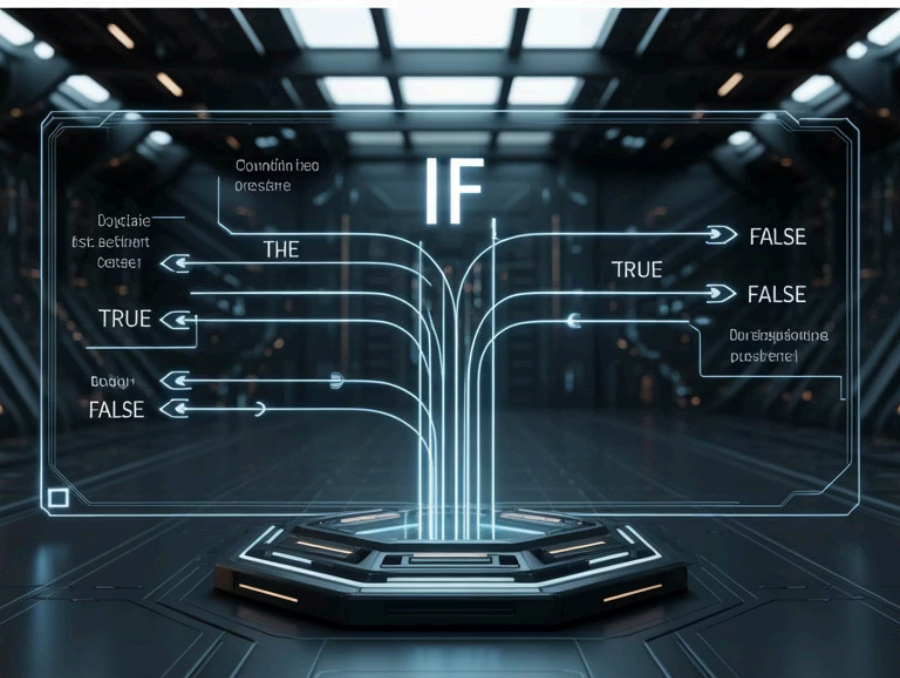
```
for i in range(5):  
    print(i)  
  
for fruit in fruits:  
    print(fruit)
```

3

While Loop

```
counter = 0  
while counter < 5:  
    print(counter)  
    counter += 1
```

Conditional statements allow for making decisions in code, while loops enable repetitive execution of operations.

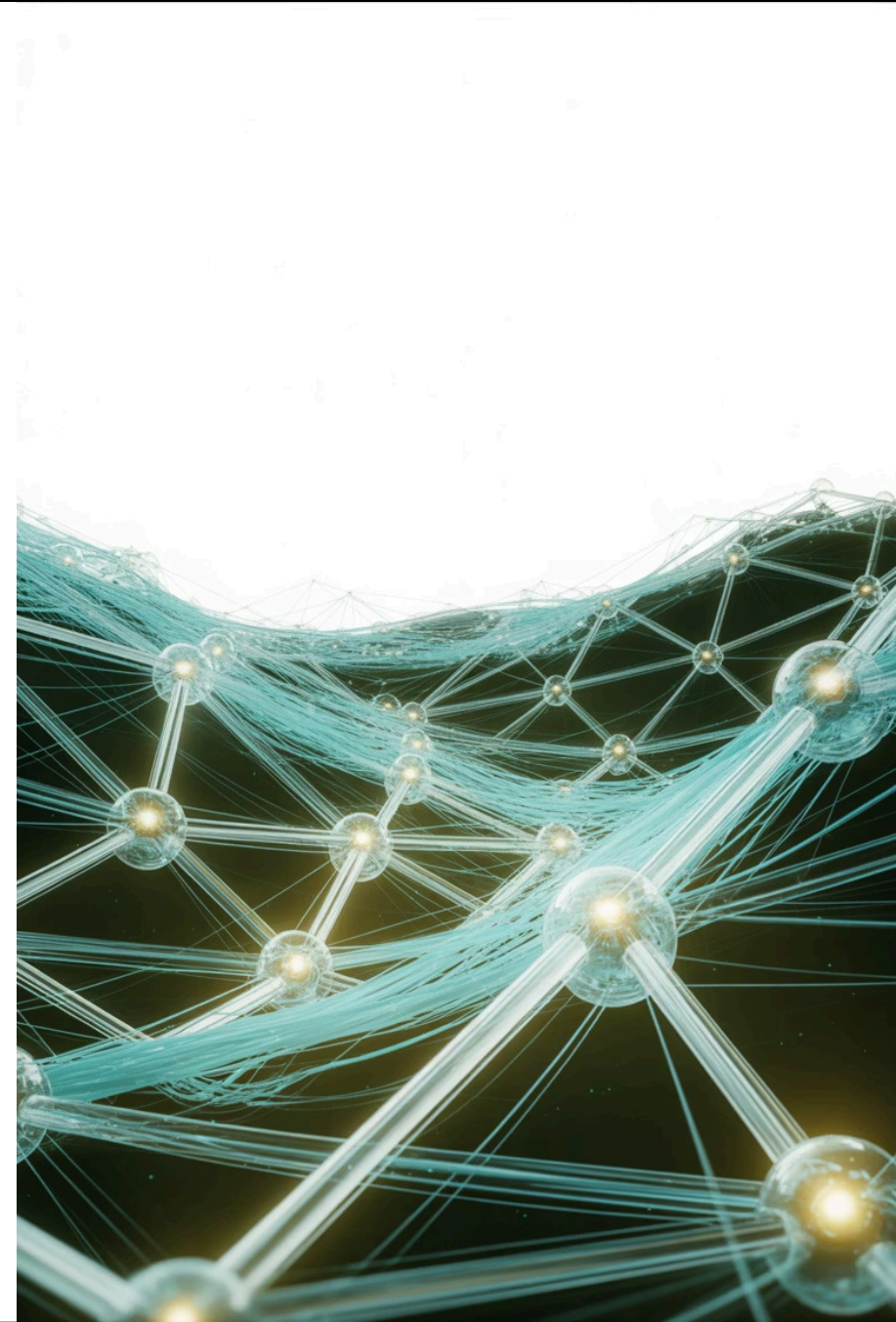


Flowchart of Conditional Statement

Chapter 3

Data Structures

Lists, dictionaries, tuples, sets



Lists (list)

Lists are ordered, mutable collections of elements. They can contain various data types and are the most commonly used structure in Python.

01

Creation and Indexing

```
fruits = ["apple", "banana", "pear"]  
first = fruits[0] # "apple"  
last = fruits[-1] # "pear"
```

02

Modifying Elements

```
fruits[1] = "orange"  
fruits.append("watermelon") # add to the end  
fruits.remove("apple") # remove element
```

03

Useful Methods

```
fruits.sort() # sorting  
fruits.reverse() # reverse  
len(fruits) # list length
```

04

List Comprehension

```
squares = [x**2 for x in range(10)]  
evens = [x for x in range(20) if x % 2 == 0]
```

Tuples (tuple)



Immutability and Uses

Tuples are similar to lists, but are **immutable** – once created, their content cannot be changed. This ensures data integrity.

```
coordinates = (52.2297, 21.0122)  
data = ("John", "Doe", 30)
```

```
# Unpacking the tuple  
x, y = coordinates  
first_name, last_name, age = data
```

Tuples are faster than lists and are often used to return multiple values from functions.

Dictionaries (dict)

Dictionaries store data in **key-value** pairs, allowing quick access to information via unique keys.



Creating a Dictionary

```
user = {  
    "name": "Anna",  
    "age": 28,  
    "city": "Warsaw"  
}
```



Adding and Modifying

```
user["email"] = "anna@example.com"  
user["age"] = 29
```



Accessing Data

```
name = user["name"]  
age = user.get("age", 0)
```



Iteration

```
for key, value in user.items():  
    print(f"{key}: {value}")
```

☐ **Practical Example:** Dictionaries are great for representing user databases, application configurations, or word counters in text.

Sets (set)

Unique Elements

Sets automatically eliminate duplicates and do not preserve element order. They are extremely fast at checking membership.

```
numbers = {1, 2, 3, 3, 4, 5} # {1, 2, 3, 4, 5}
colors = set(["red", "blue", "red"]) # {"red", "blue"}
```

```
# Adding and removing
numbers.add(6)
numbers.remove(1)
```

Mathematical Operations

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

```
A.union(B)      # {1, 2, 3, 4, 5, 6}
A.intersection(B) # {3, 4}
A.difference(B)  # {1, 2}
```

Applications: filtering duplicates from lists, checking for common elements, operations on data sets.



Chapter 4

Functions and Modules

Defining Functions

Functions are reusable blocks of code that perform specific tasks. They allow for program organization and avoiding repetition.

Basic Syntax

```
def function_name(parameter1,
                  parameter2):
    """Function documentation"""
    result = parameter1 +
            parameter2
    return result
```

The keyword `def` starts the definition,
and `return` returns a value.

Text Function

```
def greeting(name,
              time_of_day="good morning"):
    return f"{time_of_day}, {name}!"

print(greeting("Anna")) # "good
                        morning, Anna!"
print(greeting("John", "good
evening")) # "good evening, John!"
```

Mathematical Function

```
def calculate_circle_area(radius):
    pi = 3.14159
    return pi * radius ** 2

area = calculate_circle_area(5) #
                                78.53975
```

Anonymous Functions (Lambda)

Short, One-Time Functions

Lambda functions provide a concise way to create simple, anonymous functions without using the `def` keyword.

```
square = lambda x: x ** 2
print(square(5)) # 25

add = lambda a, b: a + b
print(add(3, 4)) # 7
```

Usage with `map()` and `filter()`

```
# map - applies a function to each element
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x**2, numbers))
# [1, 4, 9, 16, 25]

# filter - filters elements
evens = list(filter(lambda x: x % 2 == 0, numbers))
# [2, 4]
```

Importing Modules



math – Mathematical Operations

```
import math

print(math.sqrt(16)) # 4.0
print(math.pi)      #
3.14159...
print(math.sin(math.pi/2))
# 1.0
```



datetime – Dates and Time

```
from datetime import
datetime, timedelta

now = datetime.now()
next_week = now +
timedelta(days=7)
print(now.strftime("%Y-
%m-%d"))
```



os – System Operations

```
import os

current_directory =
os.getcwd()
files = os.listdir(".")
os.mkdir("new_folder")
```

Creating Your Own Modules: Save functions in `my_module.py` and import: `import my_module`

Error and Exception Handling

The try-except mechanism allows for **graceful handling** of errors – the program does not crash, but reacts in a controlled manner.

Basic Handling

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Multiple Exception Types

```
try:
    number = int(input("Enter a number: "))
    result = 100 / number
except ValueError:
    print("This is not a number!")
except ZeroDivisionError:
    print("Cannot be zero!")
```

Finally Block

```
try:
    file = open("data.txt")
    content = file.read()
except FileNotFoundError:
    print("File does not exist")
finally:
    file.close() # Always executed
```



Exception Handling in Practice

Safely executing code with appropriate error response



Chapter 5

Introduction to Data Processing

Data Analysis Libraries



NumPy

The foundation of scientific computing in Python. It provides efficient multidimensional arrays and mathematical functions that operate on entire datasets simultaneously.



Pandas

The most important library for tabular data manipulation. Series and DataFrame are structures that enable loading, cleaning, and analyzing data.



Matplotlib

The basic library for creating plots. It offers full control over every aspect of visualization, from axes to colors and styles.



Seaborn

A high-level library based on Matplotlib. It creates aesthetically pleasing statistical plots with simple commands, ideal for quick data exploration.

NumPy Basics

Creating Arrays

```
import numpy as np

# 1D Array
a = np.array([1, 2, 3, 4, 5])

# 2D Array (matrix)
m = np.array([[1, 2, 3],
              [4, 5, 6]])

# Special Arrays
zeros = np.zeros((3, 3))
ones = np.ones((2, 4))
range_array = np.arange(0, 10, 2)
```

Mathematical Operations

```
# Operations on entire arrays
a = np.array([1, 2, 3, 4])
print(a * 2)    # [2, 4, 6, 8]
print(a ** 2)   # [1, 4, 9, 16]
print(np.sqrt(a)) # [1.0, 1.41, 1.73, 2.0]

# Aggregations
print(a.sum())  # 10
print(a.mean()) # 2.5
print(a.max())  # 4
```

Indexing and Filtering: `a[a > 2]` will return `[3, 4]`, `m[0, 1]` will return the element from the first row, second column.

Pandas Basics

1 Series

```
import pandas as pd

temperatures = pd.Series([22, 24, 19, 21, 23],
                        index=['Mon', 'Tue', 'Wed', 'Thu', 'Fri'])
print(temperatures['Tue']) # 24
```

2 DataFrame

```
data = pd.DataFrame({
    'Name': ['Anna', 'John', 'Mary'],
    'Age': [28, 34, 29],
    'City': ['Warsaw', 'Krakow', 'Gdansk']
})
print(data.head())
```

3 Loading Files

```
# CSV
df = pd.read_csv('data.csv')

# Excel
df = pd.read_excel('report.xlsx')

# Save
df.to_csv('result.csv', index=False)
```

Data Cleaning and Preparation

Real-world data is rarely perfect. Cleaning is a crucial step in any analytical project.

1 Removing Duplicates

```
df_no_duplicates = df.drop_duplicates()
print(df.duplicated().sum()) # Number of duplicates
```

2 Handling Missing Values

```
# Check for missing values
print(df.isnull().sum())

# Remove rows with missing values
df_clean = df.dropna()

# Fill missing values
df['Age'].fillna(df['Age'].mean(), inplace=True)
```

3 Changing Data Types

```
df['Date'] = pd.to_datetime(df['Date'])
df['Price'] = df['Price'].astype(float)
df['Category'] = df['Category'].astype('category')
```

4 Filtering Data

```
# Simple filtering
adults = df[df['Age'] >= 18]

# Complex conditions
warsaw_adults = df[(df['City'] == 'Warsaw') & (df['Age'] >= 18)]
```



Chapter 6

Data Operations and Visualizations

Grouping and Aggregations in Pandas

The `groupby()` mechanism allows for dividing data into groups and performing calculations on each group separately.

Basic Aggregations

```
import pandas as pd

# Example sales data
sales_data = pd.DataFrame({
    'Product': ['A', 'B', 'A', 'B', 'A'],
    'Region': ['N', 'S', 'N', 'N', 'S'],
    'Revenue': [100, 150, 120, 200, 90]
})

# Group by product
product_group = sales_data.groupby('Product')
print(product_group['Revenue'].sum())

# Multiple groups
multi_group = sales_data.groupby(['Product', 'Region'])
print(multi_group['Revenue'].mean())
```

Various Aggregation Functions

```
# Sum, mean, count
print(product_group['Revenue'].agg(['sum', 'mean', 'count']))

# Custom functions
def sales_range(x):
    return x.max() - x.min()

print(product_group['Revenue'].agg([sales_range, 'std']))

# Different aggregations for different columns
print(sales_data.groupby('Region').agg({
    'Revenue': ['sum', 'mean'],
    'Product': 'count'
}))
```

Combining and Merging Data

1

merge() – SQL-style Joining

```
customers = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Anna', 'Jan', 'Maria']
})

orders = pd.DataFrame({
    'Customer_ID': [1, 2, 1, 3],
    'Product': ['A', 'B', 'C', 'A']
})

# Inner join - only matches
result = pd.merge(customers, orders,
                  left_on='ID', right_on='Customer_ID')
```

2

concat() – Vertical/Horizontal Concatenation

```
# Vertical concatenation (one below another)
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
result = pd.concat([df1, df2], ignore_index=True)

# Horizontal concatenation (side by side)
result = pd.concat([df1, df2], axis=1)
```

❏ **Join Types:** inner (only matches), left (all from left), right (all from right), outer (all records)

Matplotlib Visualization Basics

Matplotlib is the foundation for data visualization in Python, offering full control over charts.

01

Line Plot

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.plot(x, y, marker='o', color='blue',
linewidth=2)
plt.xlabel('Time')
plt.ylabel('Value')
plt.title('Linear Growth')
plt.grid(True)
plt.show()
```

02

Bar Chart

```
categories = ['A', 'B', 'C', 'D']
values = [23, 45, 56, 32]

plt.bar(categories, values,
color='#0069ff')
plt.xlabel('Category')
plt.ylabel('Sales')
plt.title('Sales Results')
plt.show()
```

03

Histogram

```
import numpy as np

data = np.random.randn(1000)

plt.hist(data, bins=30, color='#3d69fa',
alpha=0.7)
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Normal Distribution')
plt.show()
```

Seaborn – Elegant Statistical Visualizations

Correlation and Distribution Plots

```
import seaborn as sns
import pandas as pd

# Correlation Heatmap
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [2, 4, 5, 4, 5],
    'C': [5, 4, 3, 2, 1]
})

correlation = df.corr()
sns.heatmap(correlation, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()

# Box Plot
sns.boxplot(data=df)
plt.title('Distribution of Variables')
plt.show()
```

Scatter Plot

```
# Scatter plot with trend
sns.scatterplot(data=df, x='A', y='B',
               size='C', hue='C')
plt.title('Relationship of A to B')
plt.show()

# Pairplot - all combinations
sns.pairplot(df)
plt.show()
```

